

0.57.3. La fonction `rectangle()` — tourner en rectangle

`#include <rectangle>` : `rectangle(x, y, tuile, largeur, hauteur, sens, vitesse, tours)`, un carré qui a deux tailles.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `rectangle(...)` fait tourner la tuile en **rectangle** : `largeur` pour l'axe x, `hauteur` pour l'axe y. Largeur 4 et hauteur 2 : 9×5 cases.

La ligne à écrire : `#include <rectangle>`.

Ses arguments : huit : le centre, la tuile, la largeur, la hauteur, le sens, la vitesse, les tours.

Ce qu'elle coûte : environ **600 octets**, plus la police entière.

Essaie : une largeur de 8 et une hauteur de 1 : une piste.

On retrouve `rectangle` dans 10 leçons

[0.58](#), [0.58.1](#), [0.58.2](#), [0.62](#), [0.62.1](#), [0.62.2](#), [0.63](#), [0.63.2](#), [0.64](#), [0.64.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1440 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
rectangle()	527 o	← main() — une fonction fournie par le langage
main()	42 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	x 4	52 o sur place, dans rectangle() + AdresseCase (10 o)
effacer()	x 3	45 o sur place, dans rectangle() + AdresseCase (10 o), EffacerCases (12 o)
image()	x 4	12 o sur place, dans rectangle(), main() + AttendreImage (28 o)

```
Tes fonctions appelées :  
rectangle() x 1 35 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

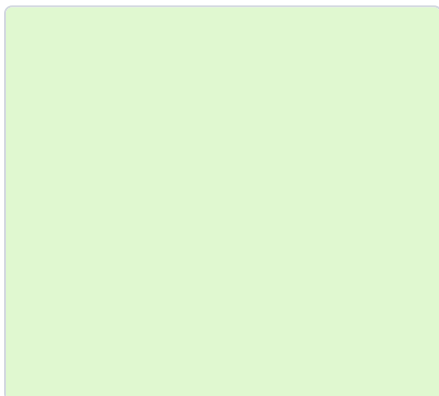
```
// ---- #include <rectangle> : tourner en rectangle ----  
  
#include <rectangle> // rectangle() : des tours en rectangle  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    rectangle(10, 8, ALPHABET[0], 4, 2, 1, 100, 1); // 9 x 5 cases autour de (10, 8)  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

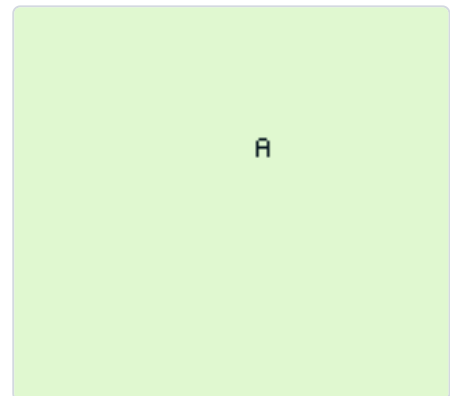
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait le tour d'un rectangle large et plat.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A touche les coins du rectangle

✓ il revient au centre

Ensuite : **0.58. Plus large que haut : rectangle** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++